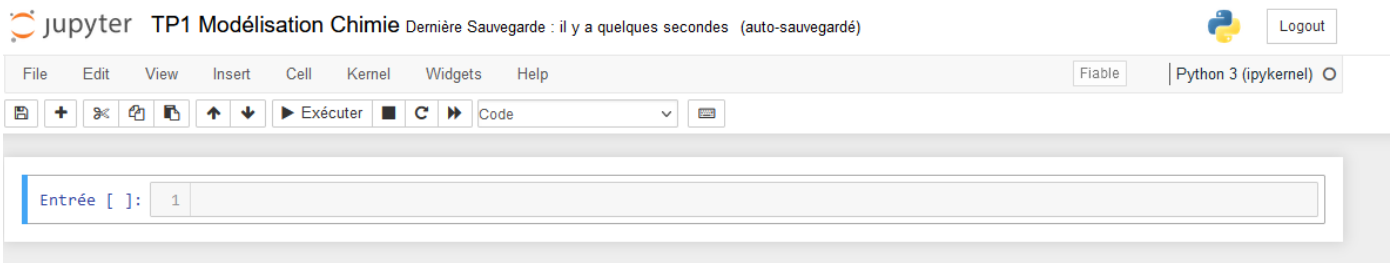
TP 1 : Introduction aux fonctions de base Python

Ce TP propose d'utiliser le langage de programmation *Python* pour le calcul et la représentation graphique d'un acide fort ou faible en solution. Il inclut l'utilisation des fonctions de base de Python (boucles, conditions et calculs mathématiques) ainsi que du module *matplotlib*.

Pour ce module, la programmation en *Python v3* (https://www.python.org/) se fera *via* la plateforme web interactive *Jupyter Notebook* (https://jupyter.org/) accessible *via* l'interface graphique *Anaconda* (https://www.anaconda.com/download). Le choix d'utiliser un langage de programmation sous licence libre (type BSD) et des logiciels open source permet une installation et une utilisation gratuites sur vos éventuels ordinateurs personnels.

1. Lancez `Anaconda Navigator` (ANACONDA NAVIGATOR)
2. Cliquez sur `Jupyter Notebook` (jupyter). Une page internet se lance.
3. Créez alors un nouveau fichier en cliquant sur `New` puis `Python 3 (ipykernel)`.
4. Nommez ce document, `TP1 Modélisation Chimie`.

Vous arrivez alors à une page où vous pouvez écrire des lignes de code ou effectuer des calculs simples pour vous entraîner. Une fois votre commande écrite, cliquez sur `Exécuter`.





Partie 1 : Calcul autour de l'acide chlorhydrique

Exercice 1 : Calcul de la concentration d'une solution d'acide chlorhydrique commercial

L'acide chlorhydrique commercial se présente sous forme d'une solution à 37 % en poids d'acide chlorhydrique.

Données : *Masse molaire :* M(HCl) = 36,46 $g.mol^{-1}$

Masse volumique : ρ(HCl commercial) = 1,19 $g.cm^{-3}$

Commande `print`.

La syntaxe pour utiliser la commande `print` est la suivante :

```
print(a)
```

Pour afficher une chaîne de caractères, utilisez des guillemets (`''`). Vous pouvez également demander à Python d'afficher le résultat sous forme scientifique et/ou avec un nombre défini de décimales en utilisant les « *f-strings* ».

Nombre de décimal défini :

```
print(f'{a:.xf}')
```

où x correspond au nombre de décimales, et f à un nombre flottant (décimal).

Ecriture scientifique avec un nombre de décimal défini :

```
print(f'{a:.xE})
```

Ici, le f est remplacé par E

Variable.

Une variable est définie avec le signe = (par exemple, `a=2`).

Attention à la casse : Python différencie les majuscules et les minuscules.

Instruction :

1. Programmez en Python le calcul de la concentration molaire en acide chlorhydrique dans cette solution. Les données et la formule de calcul seront définies comme des variables.
2. Affichez le résultat avec la commande `print`.

Notes :

……

……

……

……

……

……

……

……

……

……

<u>Exercice 2 :</u> Calcul du pH d'une solution d'acide chlorhydrique diluée.

La solution commerciale d'acide chlorhydrique à 12,076 mol.L^{-1} est maintenant diluée. Pour cela, on prélève 166 µL de solution mère que l'on complète à 100 mL avec de l'eau distillée.

Commande `def` (fonction)

Une fonction est définie à l'aide de la structuration suivante :

```
def nom_de_la _fonction (variable1,variable2,…) :
    instruction(s)  (éventuellement)
    return résultat(s)
```

Commande logarithme

La commande pour le logarithme décimal n'existe pas directement dans Python. Il faut utiliser le module `math`. Trois options s'offrent à vous :

1. Importer l'intégralité du module :

```
import math
```

2. Importer une commande spécifique :

```
from math import log10
```

3. Utiliser un alias :

```
import math as m
```

Instruction :

1. Calculez le pH de la solution fille de deux manières :
   - En définissant la concentration comme une variable.
   - En définissant une fonction.
2. Affichez :
   - La concentration fille en écriture scientifique avec deux décimales.
   - La valeur du pH avec deux décimales.

<u>Notes :</u>

……
……
……
……
……
……
……
……
……
……

Partie 2 : Calcul autour de l'acide formique

Exercice 3 : Calcul du pH d'un solution d'acide formique

Contrairement à un acide fort, un acide faible ne se dissocie que partiellement dans l'eau. La constante d'équilibre relative à cette dissociation est appelée constante d'acidité, K_a.

Donné : $pK_a(HCCOH/HCOO^-) = 3,75$

Résolution de l'équation du second degré :

Programmez la résolution de l'équation sous la forme d'une fonction `deg2(a, b, c)` en prenant en compte les trois cas :

- Δ>0 : Deux solutions réelles, on garde la valeur positive.
- Δ=0 : Une solution réelle.
- Δ<0 : Pas de solution réelle.

La prise en considération des trois cas de figure se fera à l'aide d'une structure conditionnelle.

Commande conditionnelle (`if ... elif ... else`)

La structure conditionnelle permet de programmer différent cas de figure suivant la variable mis en entrée. Son utilisation se fait par l'utilisation de si (*`if`*), sinon si (*`elif`* contraction de else if) et de sinon (*`else`*). La syntaxe est la suivante :

```
if condition:
      return instruction(s)
elif condition:
      return instruction(s)
else :
      return instruction(s)
```

Instruction :

1. Déterminez le pH d'une solution d'acide formique à $2,00 \cdot 10^{-3}$ mol.L^{-1} à l'équilibre en résolvant l'équation du second degré.

Notes :

……
……
……
……
……
……
……
……
……
……
……

Exercice 4 : Diagramme de prédominance de l'acide formique

Lorsque le pH augmente, l'acide formique se déprotone pour donner l'ion formiate. Le graphique montrant le pourcentage des deux espèces en fonction du pH s'appelle un diagramme de prédominance.

Donnée : [HCCOH] = $2{,}00.10^{-3}$ mol.L^{-1}

Boucle `for`. Une boucle *for* permet de définir une valeur de début et de fin exclue dans laquelle la boucle va être répétée. L'utilisation de *in range* permet de définir un intervalle [a,b[et un pas. Si le pas n'est pas précisé il est par défaut de 1. Il peut être augmenté mais doit nécessairement un entier.

La syntaxe d'une boucle *for* est la suivante :

```
for i in range (début, fin exclue) :
    instruction(s)
```

La boucle s'arrêtera une fois la valeur de fin atteinte. Mais les valeurs successivement prises ne seront pas sauvegardées. Pour les conserver, il est possible d'utiliser une liste qui présente l'avantage d'être mutable.

Liste Mutable.

Pour cela, une *liste* initialement vide (*x_a=[]*) est défini puis les valeurs de la boucle sont ajoutées à la liste à l'aide la commande *append* (*x_a.append(a)*). La synthaxe est la suivante :

```
x_a = []
x_a.append(valeur)
```

Commande `matplotlib.pyplot`

Pour générer un graphique, importez le module :

```
import matplotlib.pyplot as plt
```

Affichez une courbe avec :

```
plt.plot(x, y, color='...', linestyle='...', linewidth=..., label='...')
```

color (ou c) = '...', pour définir la couleur du tracé, linestyle (ou ls) = '...' pour définir le type de tracé (pointille ':', trait hachuré '--') ; linewidth (ou ld) = nombre, épaisseur du trait ; label = '...' pour définir la légende des tracés

Paramètres graphiques :

Ajoutez des titres et légendes :

```
plt.title('Titre')
plt.xlabel('Nom axe X')
plt.ylabel('Nom axe Y')
plt.legend()    # afficher la légende défini dans les paramètres graphiques
plt.show()      # demander à Python de faire le graphique
```

Instruction :

1. Créez trois listes avec :
   - Les valeurs de pH.
   - Le pourcentage d'acide formique.
   - Le pourcentage d'ion formiate.

2. Calculer à l'aide d'une boucle `for` puis stockés dans une `liste` les % en espèces acides et basiques pour un pH compris entre 0 et 14. Faites afficher par le programmateur les listes ainsi générées.

3. Réalisez une présentation graphique des pourcentages des formes acide et basique de l'acide formique en fonction du pH. Utilisez deux couleurs différentes pour distinguer les deux espèces. Choisissez des marqueurs différents pour les tracés et sélectionnez une taille de marque adaptée. Le graphique devra obligatoirement comporter un titre, des noms d'axes clairement indiqués, et les tracés des deux formes devront être facilement identifiables.

Notes :